



MATPLOTLIB

~ Arijit Das



```
import random
print(random.choice([
    "You can do it!",
    "Keep going!",
    "Believe in yourself!",
    "Stay positive!",
    "One step at a time."
]))
```

#Quote #Programming #Selfcare

{ Matplotlib }

"Picture speaks louder than a number"

History of MATPLOTLIB :

- Created by John D. Hunter in 2003 while he was working as a neurobiologist.
- Initially developed to replicate Matlab - style plotting in Python.

MATLAB + PLOT + LIB → MATPLOTLIB

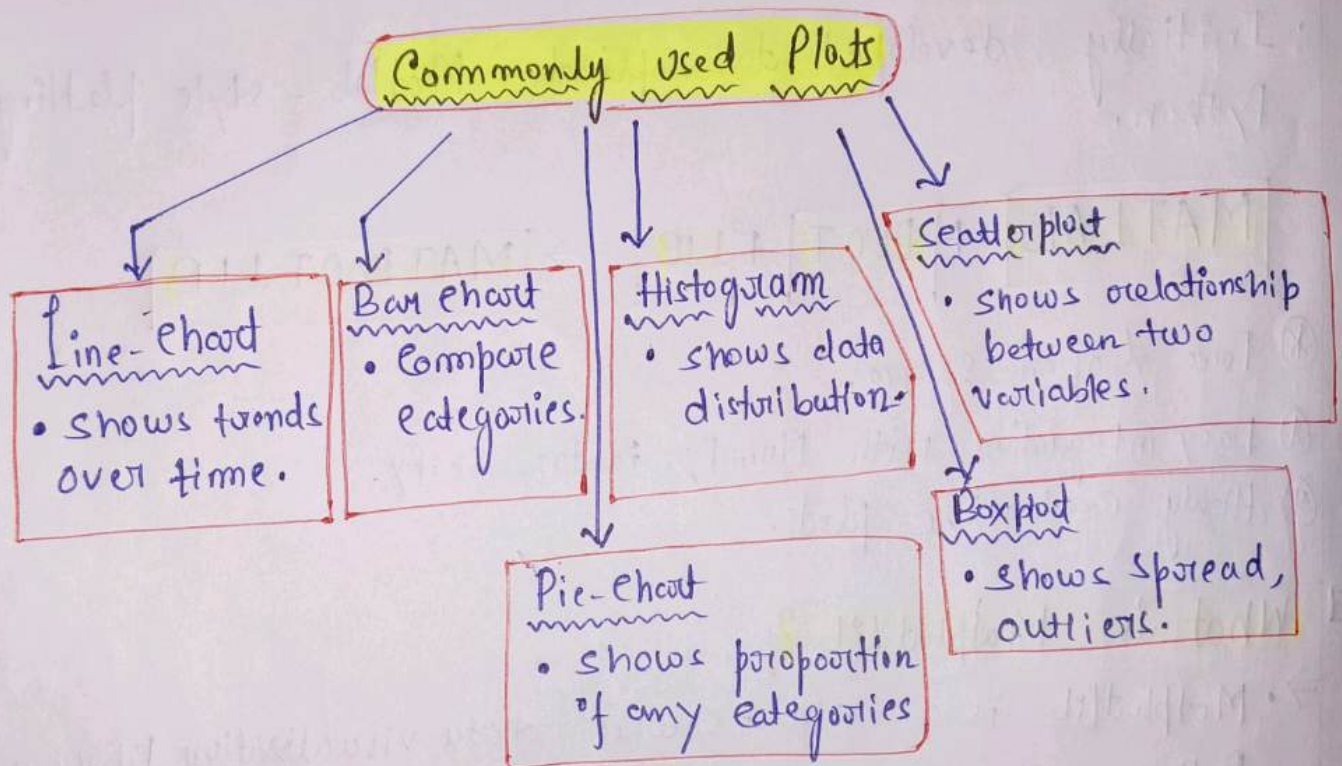
- ⊗ Free & open-source.
- ⊗ Easy integration with Numpy, Pandas, Seipy.
- ⊗ Highly customize plots.

What is Matplotlib ?

- ⇒ • Matplotlib is a open-source data visualization library in Python.
- It is mainly used in static, interactive & animated plots.
 - It provides a wide variety of charts like lineplot, bar-chart, pie-chart, box-plot, histogram, scatterplot.
 - The commonly used module is matplotlib.pyplot, usually imported as plt.

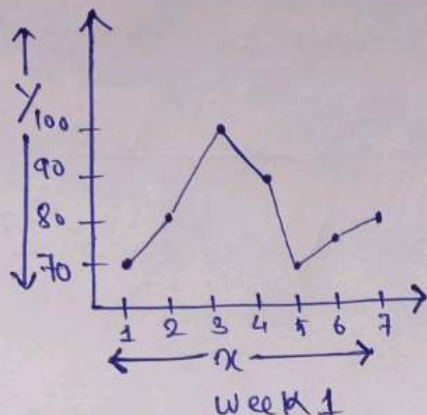
Key Features:

- Produce high quality 2-D plots.
- Supports various file formats.
- Works well with Numpy, Pandas, Sci-Py.
- Can be embedded in Jupyter Notebook, GUIs and applications.



Key Term You should know :

① **Data Point** → x -axis, y -axis.



Week 1 = [1, 2, 3, 4, 5, 6, 7]

marks = [70, 80, 100, 90, 70, 75, 80]

* Suppose you are analyzing marks of a student in a week.

* Here you have to take two data point, 1 for x axis and another for y axis.

* Here Week days is your x axis
→

* Here marks is your y axis.
↓

② **x -axis / y -axis** →

x → input

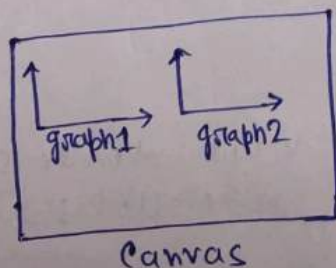
* x = Days

y → output

* y = Result

③ **Figure** →

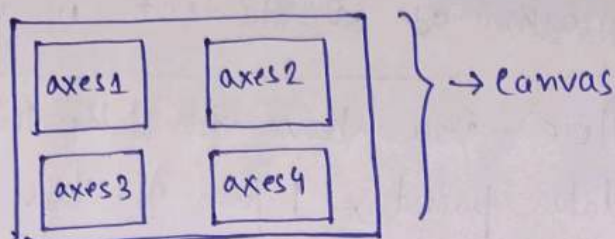
Means a canvas where multiple graph will be plotted



④ Axes

The exact location in a canvas where a graph will be plotted.

⊛ Basically it's a graph box.



⑤ Plot

The graph which you draw and you can see this by plotting.

⑥ Marker

A symbol for visualize the data points.

⑦ Line Style

The style of the line from one point to another point.

⑧ Colour

The colour of the line, marker etc etc....

⑨ Legend

The colour or the line which represents something on a graph.

⑩ Labels

We will be working with a graph where x will denote something and y will denote something, that denoting is called label.

11 Title

The name of the graph

12 Grid

The horizontal & vertical line between the graph

13 Let's draw a graph that tells week 1 temperature per day.

Step 1:

You have to import matplotlib.

import matplotlib.pyplot as plt

week = ['Mon', 'Tues', 'Wed', 'Thu', 'Fri', 'Sat', 'Sun']

temp = [35, 40, 41, 32, 35, 33, 45]

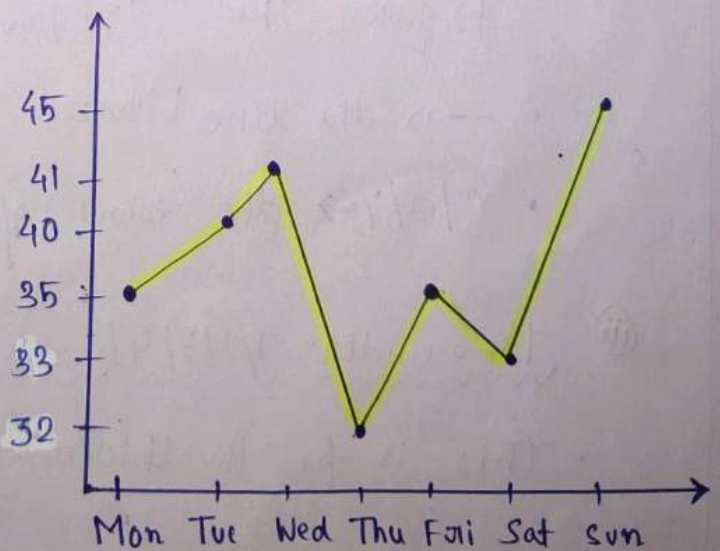
plt.plot(week, temp)

plt.title('Temperature in a week')

plt.xlabel('Week')

plt.ylabel('Temperature')

plt.show()



PyPlot functions :

■ Why we are using it?

- i To convert numbers into pictures.
- ii To draw graphs.
- iii To draw patterns through the numbers.

■ Functions :

i `plt.plot()`

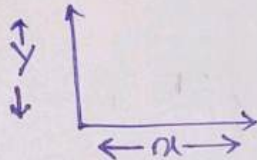
⇒ we use `plt.plot()` functions to connect one data point to another data point.

■ It takes many arguments.

↳ Main two arguments are (x, y)

`X` ⇒ X-axis

`Y` ⇒ Y-axis



• `plt.plot(x, y)`

ii `'g--' / 'r--' / 'y--' / ...`

↳ green Red yellow

• `--` ⇒ the line between two point.

• `g/r/y` ⇒ the color of the line.

iii `Line width = 1/2/3/4 / ...`

• This is for the thickness of the line.

`plt.plot(x, y, 'r--')`

`plt.plot(x, y, 'r-',
linewidth=1)`

(iv) marker :

'o' → The points shape. | `plt.plot(x, y, linewidth=2, marker='o', linestyle='--')`

(v) xlabel :

This is used for giving the title of x-axis, horizontally.

`plt.xlabel(' ')`

(vi) ylabel :

This is used for giving the title of y-axis, vertically.

`plt.ylabel(' ')`

① Now let's draw a graph, where a sales data is given.

`x = [1, 2, 3, 4, 5]`

`y = [18000, 20000, 30000, 40000, 15000]`

`plt.title('Sales data of 5 months')`

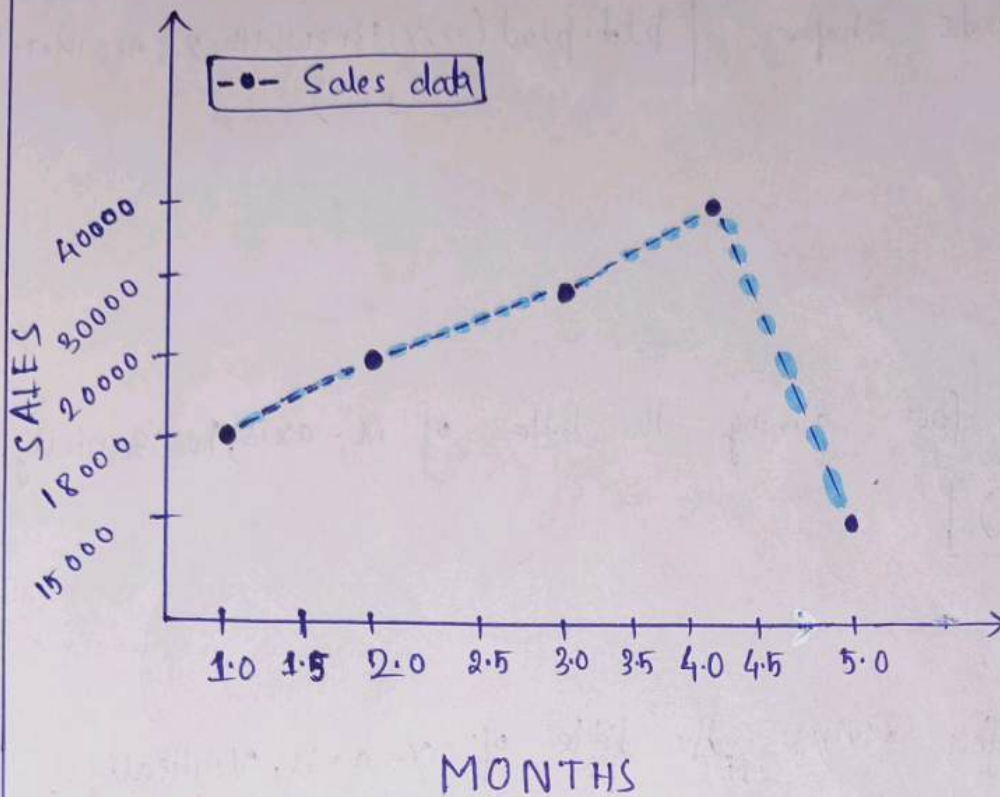
`plt.plot(x, y, 'b--', linewidth=2, marker='o')`

`plt.xlabel('MONTHS')`

`plt.ylabel('SALES')`

`plt.show()`

Sales data of 5 months



vii) Label :

This will add a title / legend inside your graph

```
plt.plot(x, y, 'b--', linewidth=2, marker='o', Label = 'Sales data')
plt.legend()
```

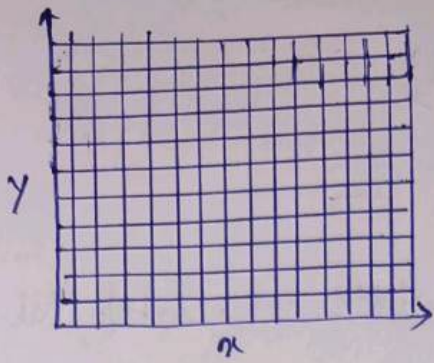
viii) Grid :

This will add lines behind your graph for better readability.

```
plt.grid()
```

■ Also you can pass parameters inside grid.

```
plt.grid( color='black', linewidth=1, linestyle=':')
```



plt.grid()

Bar Graph / Bar Chart

- Bar graph represents categorical data.
like → (Products, Departments, Regions)

Syntax:

`plt.bar(x, y, colour='colour name', width=0.6, label='label name')`

`x` = Product / Dept / Region etc etc.

`y` = Sales / Some numerical data information.

Example :

```
import matplotlib.pyplot as plt
```

```
product = ['Soap', 'Shampoo', 'Oil']
```

```
sales = [1000, 2000, 1500]
```

```
plt.bar(product, sales, width=0.6, colour='Blue', label='Sales')
```

```
plt.title('Total sales analysis')
```

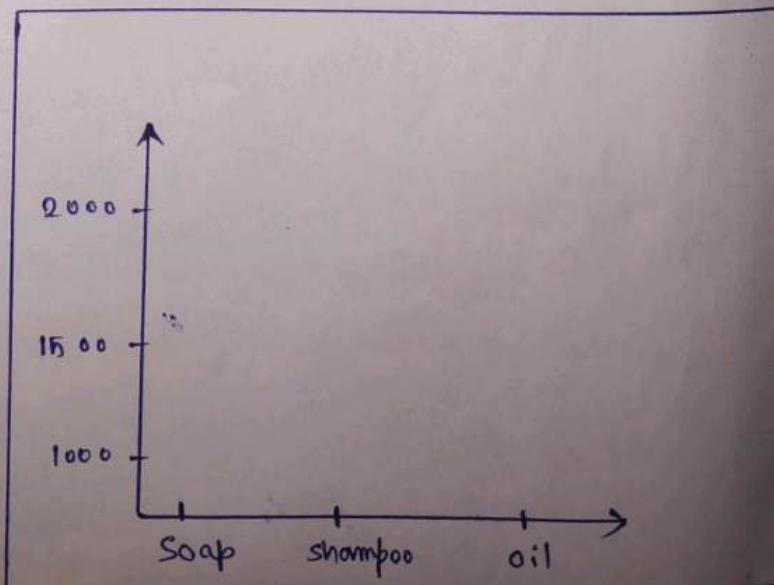
```
plt.xlabel('Product')
```

```
plt.ylabel('Sales')
```

```
plt.grid(colour='Black', linestyle=':', width=1)
```

```
plt.legend()
```

```
plt.show()
```



* Also you can change the color / fix the color for each bars.

color = ['Blue', 'Green', 'Yellow']

plt.bar (Product, Sales, width=0.6, color=color, label='Sales of month')

Pie-chart

- Pie-chart helps us to show proportion of Categories
→ Good when you want to see "how much each category contributes to the total".



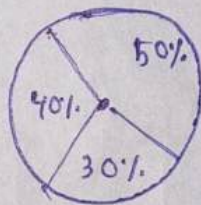
→ Example :

Market share of Companies.

A Company = 40%

B Company = 50%

C Company = 30%



→ It highlights dominant categories.

↳ which category have maximum amount.

↳ which category have minimum amount.

Syntax :

plt.pie(x, y)

plt.show()

You can add other arguments also as features.

Example 0

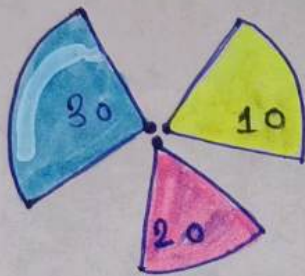
```
product = ['Apple', 'Samsung', 'Nokia']
```

```
value = [30, 10, 20]
```

```
ex = [0.1, 0, 0.1]
```

```
plt.pie(value, labels=product, shadow=True, explode=ex)
```

```
plt.show()
```



Here,

- `Shadow = True`, adds shadow under the chart
- `explode` helps us to find out the maximum amount & minimum amount.

Box plot

Purpose:

The main purpose of boxplot is to show the IQR (Interquartile Range) - the spread of the middle 50% of the data.

• Box (The rectangle):

→ Represents the IQR (50% middle spread)

→ The middle 50%. $[Q_1 \rightarrow Q_3]$ $[25\% \rightarrow 75\%]$
 (Q_1) (Q_3)

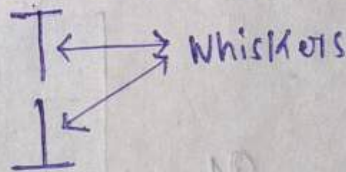
• Line inside the box:

That represents Q_2 [The middle 50%]

$Q_1 \rightarrow Q_2 \rightarrow Q_3$
 (25%) (75%)

• Whiskers:

Lines extending from the box (outside the box), from Q_1 and Q_3 . From min to max.



• Dots beyond the whiskers:

The outliers are called dots.

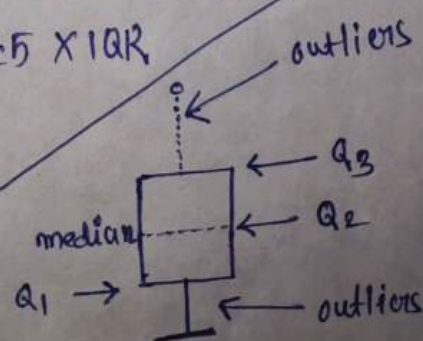
Explanation:

• Whiskers usually extend upto $1.5 \times IQR$

• $IQR = Q_3 - Q_1$

• upper limit: $Q_3 + 1.5 \times IQR$

• lower limit: $Q_1 - 1.5 \times IQR$



Syntax :

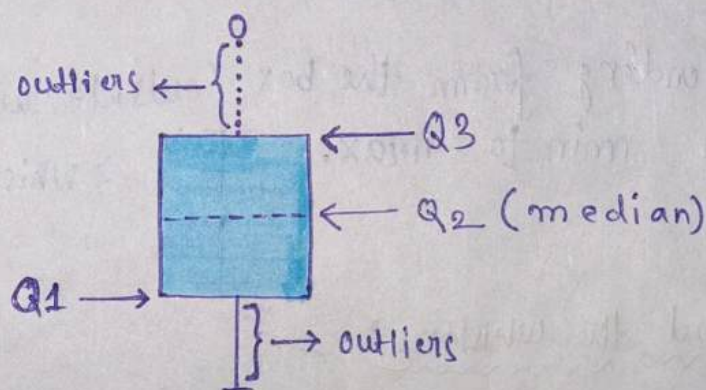
```
plt.boxplot(data, vert=True, patch_artist=True, tick_labels=['  
plt.show()
```

data : The numeric value which we are passing.

vert : The box direction (Vertical = True/False).

patch_artist : The color inside the box (True/False).

tick-labels : The x-axis title.



Box-plot

Error bar plot

Def :

- `plt.errorbar()` is a function used to plot data points along with their variability (or uncertainty)
- Blue points → Actual number of values per year.
Red lines → error bars showing the SD. (vertical)

Why we use it?

- Basically we use it for showing the Standard Deviation.

Syntax :

```
plt.errorbar(x, y, yerr=std, fmt='o', capsize=5, ecolor='red',  
            color='Blue', label='label')
```

Arguments :

`x` = The x-axis input value.

`y` = The y-axis output value.

`yerr` = The standard Deviation.

`fmt` = format of the data points ('o', '-') / `marker`=('o', '-')

`capsize` = Size of the horizontal caps.

`ecolor` = Color of the error bars.

`color` = Color of the data points.

`label` = The subject of that all arguments.

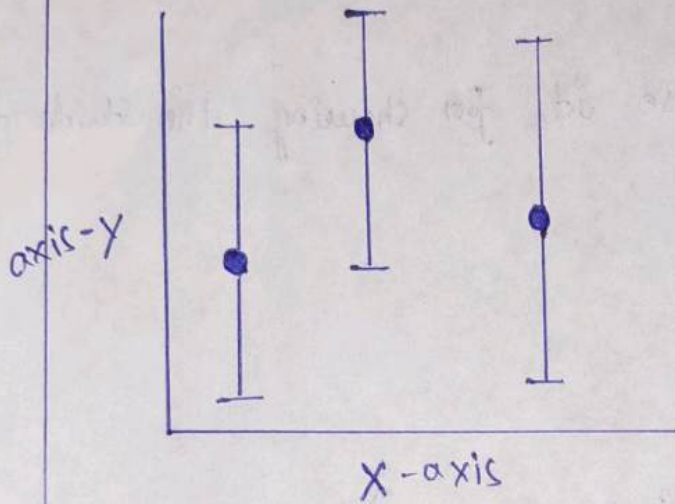
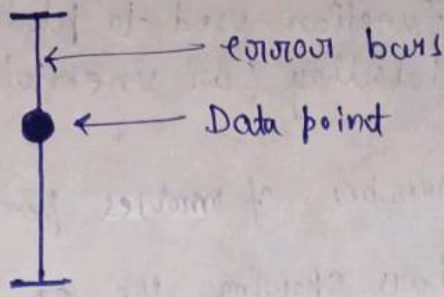


Figure 1: The effect of data dispersion

Figure 2: The effect of data dispersion

Figure 3: The effect of data dispersion

Figure 4: The effect of data dispersion

Figure 5: The effect of data dispersion

Figure 6: The effect of data dispersion

Figure 7: The effect of data dispersion

Figure 8: The effect of data dispersion

Figure 9: The effect of data dispersion

Histogram

Defn • A histogram is a graph that shows the distribution of the data.

• It looks like bar chart but instead of showing categories in x-axis, it shows ranges.

• X-axis: The bins or ranges (interval) of the data.

• Y-axis: How many data points fall into each ranges.

Syntax:

```
plt.hist (data, bins=None, edgecolor='black', color='blue')  
plt.show()
```

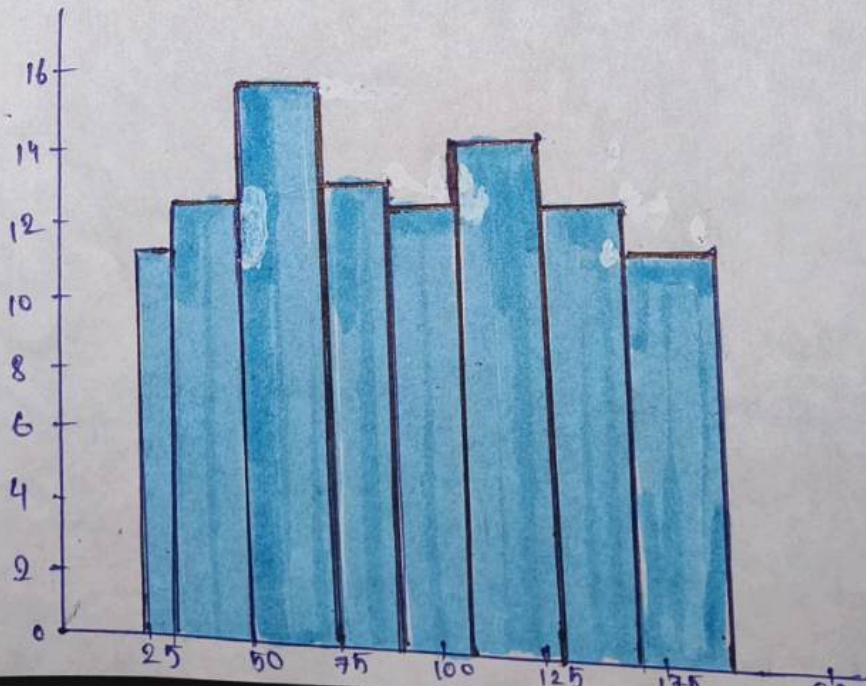
Example:

```
import numpy as np
```

```
data = np.random.randint(10, 201, size=100)
```

```
plt.hist(data, bins=10, edgecolor='black', color='blue')
```

```
plt.show()
```



Scatter plot

Defⁿ scatter plot is a graph that represents the relationship between two variables using dots.

- X-axis represents the input variable.
- Y-axis represents the output variable.

Example:

If you have students study hours and their exam marks, you can put it:

X = will represent the study hour (input)

Y = will represent the score (output)

■ Why we need it?

- Relationship correlation.
- Strength of relationship.
- Direction.
- Detect outliers.

① Relationship / correlation:

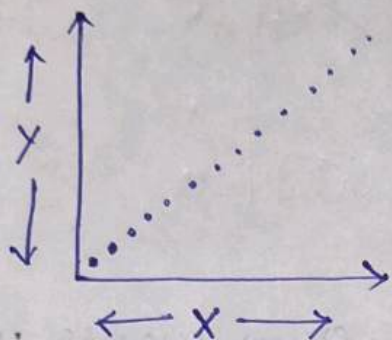
- more hours to study, more marks.

② Strength of relationship:

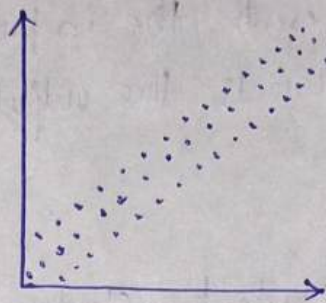
- Strong → If all the dots are close to the line.
- Weak → If all the dots are spread out.

Direction:

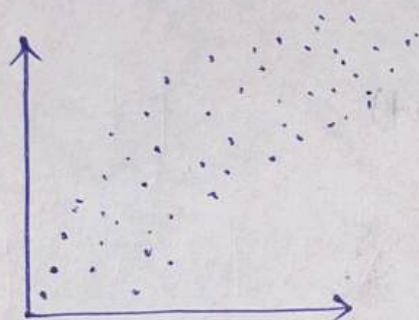
• Positive:



Perfectly
Positive

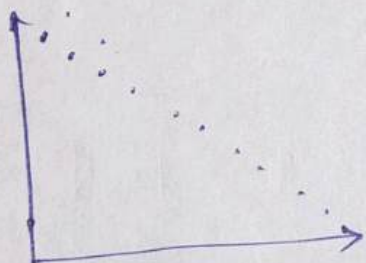


Highly
positive

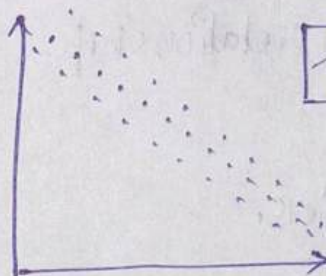


Low
positive

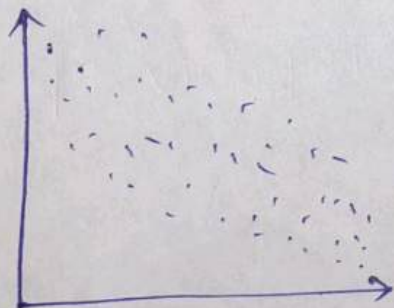
• Negative:



Perfectly
Negative



Highly negative

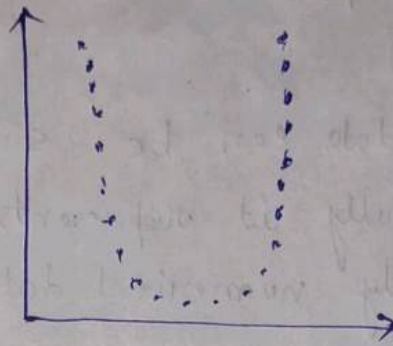


Low Negative

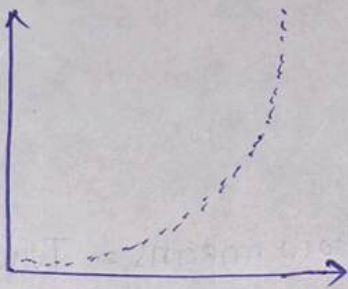
Null :



V shaped :



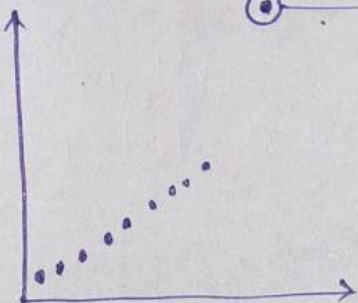
exponential :



④

Direct outliers :

⊙ → outlier



Syntax

```
plt.scatter(x, y, marker='o', color='b', label='label name')
```

```
plt.show()
```

Violin plot

Defn

• violin plots are used to see the distribution of the data.

- The data can be sales, marks, height.
- Basically it represents the y-axis output values.
- Mainly numerical data.
- That represents how the values are spread out across the x-axis.

Syntax:

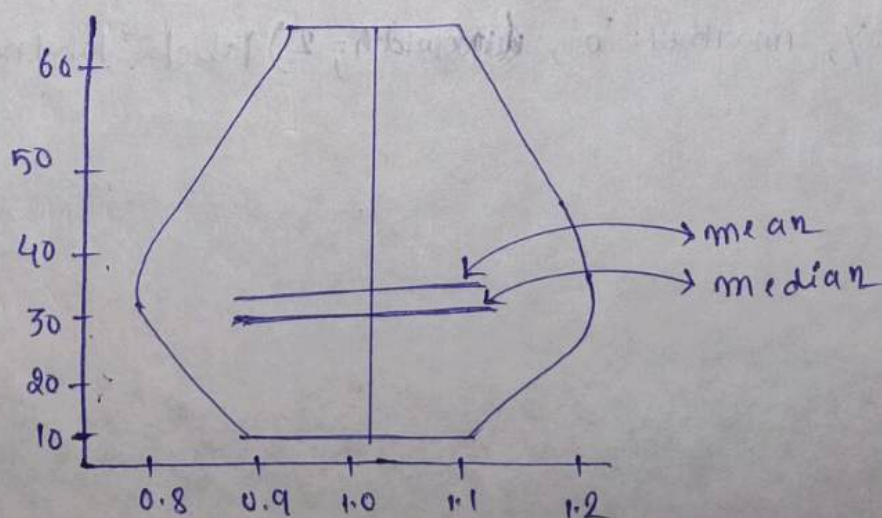
```
plt.violinplot(Y, showmedians=True, showmeans=True)
```

- showmedians = It shows the exact middle point value.
- showmeans = It shows the average of the dataset.

Example:

```
Sales = [10, 20, 30, 30, 30, 50, 60]
```

```
plt.violinplot(Sales, showmedians=True, showmeans=True)
```



Stemplot

Defn We use Stemplot to visualize the distribution of the data.

Usage :-

- (i) Helps you to see the distribution of the data.
 - (ii) Helps to see the outliers.
 - (iii) Compare different datasets quickly if multiple stemplots are drawn side by side.
- Helps to see the highest value and lowest value, and quartiles and small data distribution.

Syntax :-

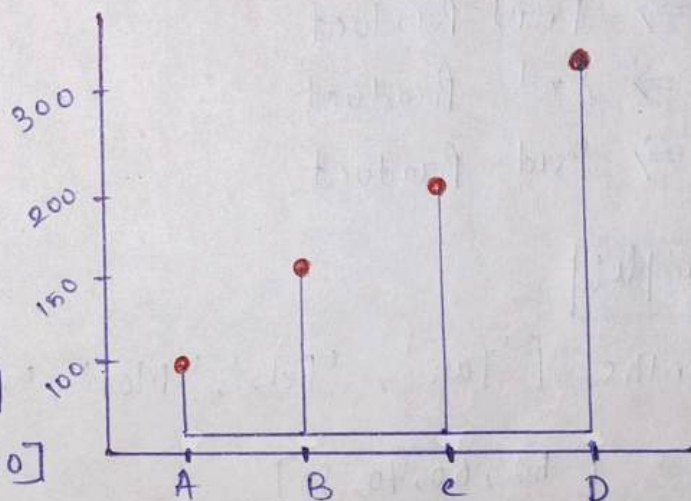
```
plt.stem(X, Y, markerfmt='o', label='label name')
```

```
plt.show()
```

- x = product
- y = Sales

example :-

```
product = ['A', 'B', 'C', 'D']  
sales = [100, 150, 200, 300]
```



```
plt.stem(product, sales, markerfmt='o')
```

```
plt.show()
```


Stack plot

Defn

A stack plot in data visualization is used to show how different components contribute to a whole over time or categories.

Usage

- i) visualize trends over time.
- ii) highlight cumulative effect.

Syntax:

```
plt. stackplot(Months, p1, p2, p3)
```

```
plt. show
```

month = months

p1 $\Rightarrow$ First Product

p2 $\Rightarrow$ 2nd Product

p3 $\Rightarrow$ 3rd Product

Example:

```
months = ['Jan', 'Feb', 'Mar', 'Apr']
```

```
p1 = [50, 60, 70, 80]
```

```
p2 = [30, 40, 50, 60]
```

```
p3 = [20, 30, 40, 50]
```

```
plt. stackplot(months, p1, p2, p3)
```

```
plt. show()
```